Автогенерация документации, попробуем обобщить

Николай Поташников

к.э.н., Архитектор решений, КУРС-ИТ

consulting@yandex.ru @nmpotashnikoff

techwriter days

> *Вся прекрасная музыка должна быть первоклассной и всегда соответствовать абсолютным правилам музыки, которые не должны нарушаться или предписываться*

— Лист Ференц

О чем этот доклад

- Технические писатели больше не нужны (нужны скрипты, LLM-агенты, хорошо написанный код, …) — **НЕТ**
- О технических решениях для автогенерации документации — **частично**
- Что дает автогенерация документации техническому писателю и другим ролям в команде — **частично**
- Когда автогенерация дает эффект, а когда о ней лучше не думать — **ДА**

Источники для автогенерации документации

Нужно сделать/обновить документацию на незнакомый продукт, а помочь некому

Что мы можем изучить, чтобы понять, как это работает

- Код
- Модульные тесты
- Артефакты сборки
- Результаты выполнения артефактов сборки
- Интеграционные/e2e-тесты

Виды создаваемых артефактов

Только сниппеты/готовые части документации?

Что не так с созвездием Кассиопея?

Автогенерация
документации, попробуем
обобщить

techwriter days

А если не те константы i18n?

```
{
  "attempt-result": {
    "detailed-results-btn" : "Детализировать результаты"
  }
}
```

```
Для детализации результатов нажмите button:[attempt-result.detailed-results-btn].
```

Автогенерируемый артефакт для документации может определять ключевые элементы документа/раздела

Промежуточный итог

Источники документации	Виды создаваемых артефактов	Что с этим делать?
• Код • Модульные тесты • Артефакты сборки • Результаты выполнения артефактов сборки • Интеграционные/e2e-тесты	• Сниппеты (готовые части документации) • Ключевые элементы документа/раздела	~~Рецепты~~ Условия

Автогенерация документации, попробуем обобщить

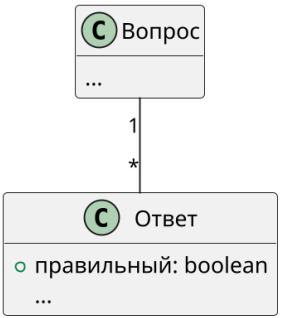
DDD — единый язык описания домена

Как описать логическую модель теста с вариантами ответов?

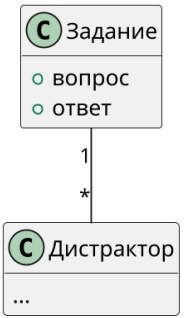
Расшифруйте DDD в названии слайда?

- Documentation Driven Drinking
- Domain Driven Design
- Deadline? Documentation Delays!

Удобно для реализации или удобно для эксперта (тестолога)?



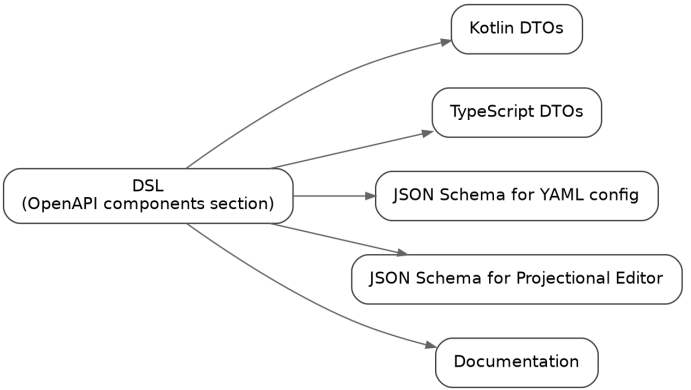
vs



Everything as code (все как код)

- Единообразие инфраструктуры разработки
- Единообразие процессов разработки
- Условия для создания коллаборации

DSL (предметно-ориентированный язык) как источник правды



* пример взят из https://fosdem.org/2026/schedule/event/W99UA3-dsl-to-docs/

Далеко ходить не надо

В моих материалах доклада на FOSDEM`26 (https://fiddlededee.github.io/fosdem-printing)

Available Gradle tasks

- **checkSpelling**: Checks spelling
- **fodt2All**: Converts notes from FODT (LibreOffice) to all formats, needs FODT file, created with makeOD, needs LibreOffice and Kotlin installed
- **makeHTML**: Creates HTML version of presentation
- **makeIndex**: Compiles index file
- **makeOD**: Creates OD (FODT) version of presentation with notes
- **makePDF**: Creates PDF from HTML version of the presentation

Как это сделано?

Gradle — система сборки проектов (в частности, данной презентации)

Встроенный в Gradle DSL для документации

```
group = "talk"
description = "Compiles index file"
```

```
tasks
    .filter { it.group == "talk" }
    .joinToString("\n") { "* *${it.name}*: ${it.description}" }
    .let { File("tasks.adoc").writeText(it) }
```

Небольшой кейс

Поставщик сервиса выдал нам спецификацию OpenAPI, однако нам нужно документировать, каким образом наше решение вызывает API и обрабатывает результат?

Вариантов множество, был бы DSL

Использование тестов

В тестах проще разобраться, чем в коде

Пример — TeX-бэкенд для Asciidoctor

UniDoc Publisher (https://github.com/fiddlededee/unidoc-publisher) — мой Pet-проект для конвертации любой разметки в печатные форматы представления

Листы у меня в руках с замечаниями к презентации — результат использования UniDoc Publisher в проекте текущей презентации (PDF получен через LaTeX)

Документация на TeX DSL

- Для создания TeX-бэкенда потребовалось создать TeX DSL
- Все тесты были разделены на документационные и регрессионные
- Хотя LLM ничего не знают об этой функции (она появилась месяц назад), он легко генерирует необходимый код

Вид теста

```
@Test
fun group() =
    tex {
        group {
            macro("color") { parameter(Brackets.REQUIRED, "") { add("orange") } }
            macro("itshape")
            +"smth"
        }
    }.toString() isEqualTo
            """
                {\color
                  {orange}%
                  \itshape
                  smth%
                }%
            """.trimIndent()
```

Автогенерация
документации, попробуем
обобщить

Результат

Markdown	2004
reStructuredText	2002
AsciiDoc	2002
MediaWiki	2002
BBCode	1998
Org-mode	2003
Textile	2004
Creole	2007
POD	1994
Setext	1992

e2e-тесты еще лучше

```
login(studentCredentials)
setPersonalInfo()
NavigationDrawer.showMenu()
NavigationDrawer.attemptGrid.click()
AttemptGridPage.newSelftestAttemptBtn.click()
screenshot("quiz--setup-phase--new-attempt-dialog")
NewSelftestAttemptDialogue.packageItem("FoodHygiene_2025_v2").click()
AttemptPage.apply {
    showAttemptInfo shouldBe visible
    attemptPageAnswers shouldBe visible
    screenshot("quiz--testing-phase--question-view")
    ...
```

Автогенерация
документации, попробуем
обобщить

Особенности использования LLM

- Тест дает скриншоты и снэпшоты HTML. Скриншоты идут напрямую в документацию и LLM может сравнить описание с интерфейсом
- По каждому несоответствию проводится оценка по 10-балльной шкале
- Что бы мы не исправляли и как бы не улучшали промпт, 10 будет всегда
- Почему? В интерфейсе написали «Не пройденные задания»: либо в документации будет грубая орфографическая ошибка, либо она не будет соответствовать интерфейсу

Используется метрика — количество несоответствий с оценкой 10. Если это число уплывает, можно поставить задачу техническому писателю — вернуть в рамки

Чем интересен данный пример

- Показанный подход (Page Objects) избыточен, но позволяет легко прочесть тест, например, ручному тестировщику
- Тестовые данные дают адекватные скриншоты для документации? Зачем это тестировщику?
- Мы должны думать, чтобы снэпшот HTML был понятен для LLM. Зачем это разработчику?

Краткий итог

- Единый язык описания домена
- Everything as code (все как код)
- Domain specific language (DSL, предметно-ориентированный язык программирования)
- Использование тестов, особенно e2e-тестов

Выводы

- Автогенерация документация требует вовлеченности всех участников проекта
- Автогенерация документация практически невозможна без хорошей архитектуры проекта

> *Вся прекрасная музыка должна быть первоклассной и всегда соответствовать абсолютным правилам музыки, которые не должны нарушаться или предписываться*
>
> *— Лист Ференц*

Вопросы?

@NMPOTASHNIKOFF